

2025 University/College IC Design Contest

Cell-Based IC Design Category for Undergraduate Level

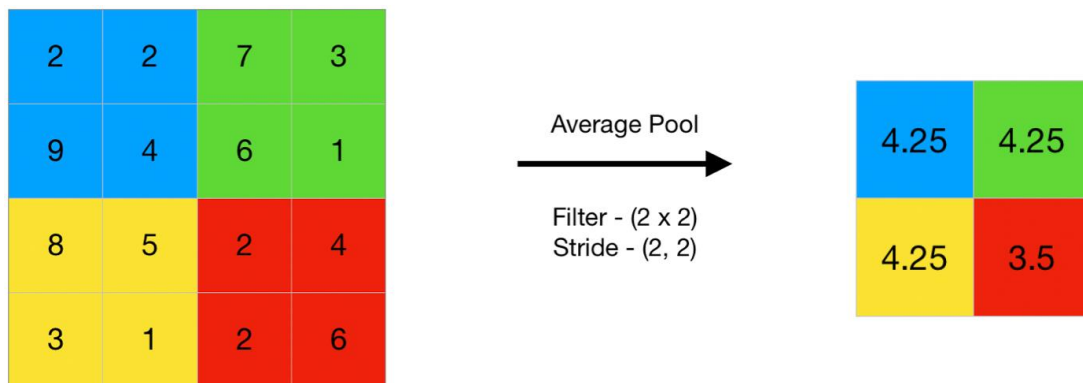
Hamming Code and Pooling

1.問題描述

Pooling(池化)是神經網路中的一道程序，可以在壓縮原始資料的同時，保留重要的特徵。

請實現一個 Pooling 系統（以下簡稱 **POOL**）。該電路會接收 81 筆(9x9)經 Hamming code 的資料，對其進行錯誤碼更正後，進行 **average pooling** 處理壓縮成 9 筆資料(3x3)。並對 pooling 處理後的結果進行 Hamming code 編碼傳回。

本次 IC 設計競賽比賽時間為上午 08:30 到下午 20:30。當 IC 設計競賽結束後，會根據第三章節中的評分標準進行評分。為了評分作業的方便，各參賽隊伍應參考附錄三中所列的要求，附上評分所需要的檔案。



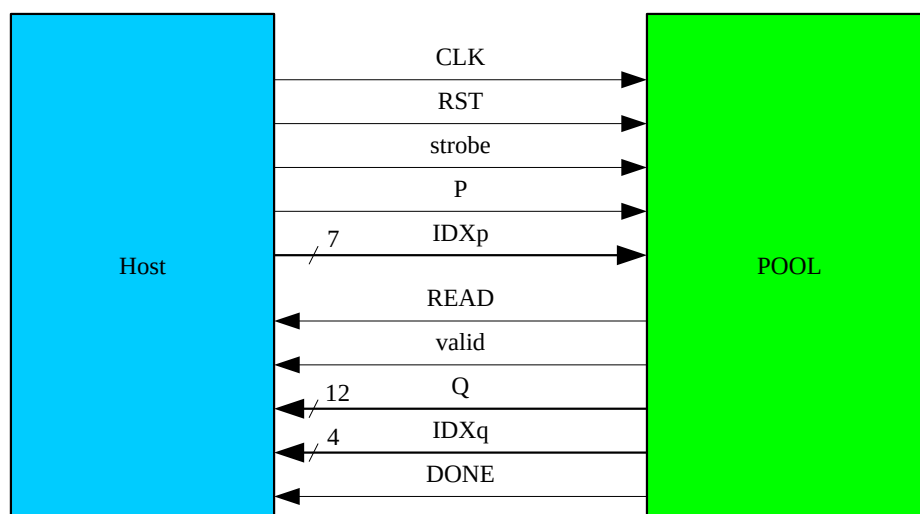
圖一、Pooling 示意圖

本題目之測試樣本置於 [/cad/icc2025/euc/e2025.tar](#)，請執行以下指令取得測試樣本：

```
tar xvf /cad/icc2025/euc/e2025.tar
```

2.設計規格

2.1 系統方塊圖



圖二、系統方塊圖

2.2 輸入/輸出介面

表 1 -輸入/輸出訊號

Signal Name	I/O	Width	Simple Description
CLK	I	1	本系統為同步於時脈正緣之同步設計
RST	I	1	高位準非同步(active high asynchronous)系統重置信號
strobe	I	1	輸入致能(strobe)控制訊號，當 strobe 為 high (1)時，輸入的 P、IDxp 方為有效
P	I	1	新資料之輸入埠。以序列方式分 12 個 cycle 輸入該資料的 12 個 bit Hamming code。此 Hamming code 有可能帶 1-bit error。
IDxp	I	7	輸入 P 資料的線性 id 編號
READ	O	1	POOL 要求輸入新資料
valid	O	1	輸出有效(valid)控制訊號，當 valid 為 High (1)時，輸出訊號 IDXq、Q 方為有效。 非輸出狀態時，valid 要求為 low(0)
Q	O	12	計算完的 Pooling 結果輸出的 Hamming code，為 12 bit 非輸出狀態時，Q 要求為 12'b0 (0)
IDXq	O	4	輸出 Q 資料的 id 編號 非輸出狀態時，Q 要求為 4'b0 (0)
DONE	O	1	初始值為 1'b0。當 POOL 電路完成所要求工作時(包括輸出 Q、IDXq 值)，該訊號拉高 1 周期，告知 Host 端可以開始評分。

2.3 系統描述

2.3.1 系統功能

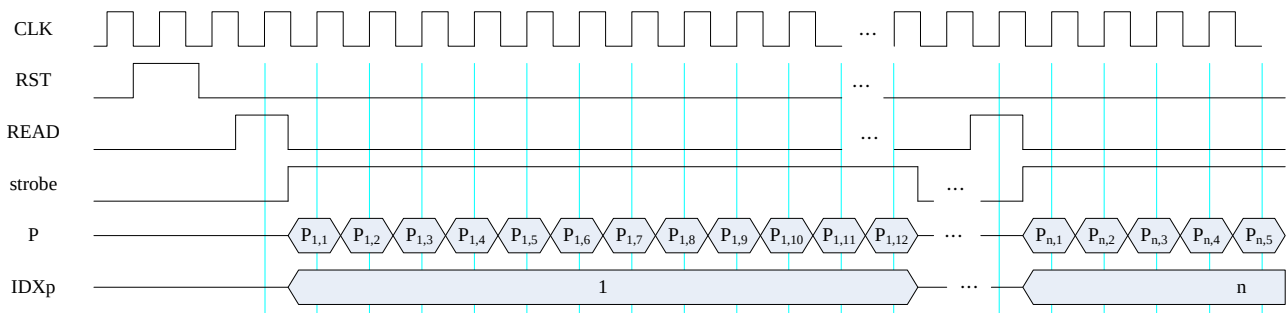
POOL 要處理的資料為 9x9 矩陣大小(81 筆)，每個點的值域為 8-bit 正整數(0~255)。為改善通訊問題，以其 12-bit Hamming code 作為 POOL 的輸入。

但 HOST 傳入 POOL 的資料因通訊雜訊問題，有可能在傳送時產生最多 1-bit 的 error (此處不考慮更惡劣的狀況)。即 HOST 傳入 POOL 的資料是 8-bit 原始資料的 Hamming code (12-bit)，且這 81 筆 12-bit 資料中，有的是正常的，有的會帶 1-bit 的 error。

因此，POOL 電路要先處理輸入資料，將接收到的 81 筆 12-bit Hamming code 讀入，並驗證每筆 Hamming code 的正確性，以及必要時(發現帶 1-bit error 時)對該筆資料進行更正 (correct)，並由正確的 12-bit Hamming code 還原回正確的 8-bit 輸入資料。

自 POOL 電路獲得要處理的 81 筆 8-bit 正確輸入資料後，依 average pooling 的方式將其處理成 3x3 的矩陣資料。並對該計算結果的 8-bit 3x3 矩陣資料(9 筆)各別進行 Hamming code 編碼，再依題目要求的方式輸出到 HOST。

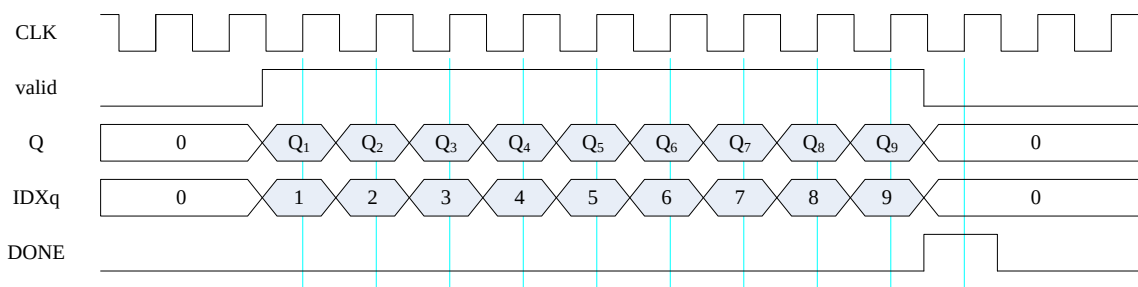
2.3.2 資料輸入



圖四、資料輸入波形

1. 系統 reset 後 POOL 即可拉高 READ 1 cycle 向 Host 要求輸入一筆 Hamming Code 資料。
2. Host 接收到 READ 要求後，接著 12 個 cycle 會以序列傳送的方式從 P 傳送該筆 Hamming Code 資料，由 LSB 到 MSB 依序送出該筆 Hamming Code 資料的各 bit。這 12 個 cycle 期間，strobe 會維持在 HIGH，IDXp 傳送該筆資料的線性 id 編號(即「第幾筆」資料)。
3. READ 拉高後的 12 個 cycle 期間，Host 會暫停接受 POOL 讀取新資料的要求。
4. 輸入資料共 81 筆，由 IDXp = 1 依序到 IDXp = 81。POOL 向 HOST 要資料的時間點可自行決定，由 POOL 用 READ 訊號拉高 1 cycle 來要求。

2.3.3 結果輸出



圖五、結果輸出波形

1. POOL 電路計算完成後，將 valid 訊號拉高為 high 連續 9 個 cycle 後再拉回 low，valid 拉為 high 代表該時間輸出的 IDXq、Q 為有效。
2. POOL 電路的輸出訊號應配合 valid 訊號，依序輸出 Q1、Q2、Q3、Q4、Q5、Q6、Q7、Q8、Q9。同時 IDXq 會輸出對應的計算結果線性 id 編號(即「第幾筆」結果)。
3. 非輸出時(valid 為 low)，Q 跟 IDXq 均要拉回 low。
4. 當 POOL 電路計算完並將所有結果輸出後，POOL 電路會在輸出 Q9 的下一個周期將 DONE 訊號拉為 high 一周期，然後拉回 low，testbench 依此訊號來判斷輸出是否完成(若 DONE 未拉為 high 則不予計分)。
5. 以 DONE 拉起來的時間點(cycle 數)為模擬時間長度(會顯示在 simulator console)，以進行評分。

Simulation 時統一使用 **周期 6 ns** (166.67MHz)作為電路之工作時脈。Testbench 預設整個 simulation 最多耗費 50000 周期，會在 50000 周期處發出\$finish 強制停止 simulation。若有需要更多的周期進行 simulation，需自行修改 tb.sv 的 MAX_CYCLE 值(約第三行位置)，並將所使用的 MAX_CYCLE 值填寫到評分表內。

2.3.4 記號方式

輸入資料為共 81 筆資料，對應到 9x9 矩陣。在輸入階段對應的 IDXp 依序為 1、2、3、4、...、79、80、81，順序不可更改。IDXp 對應到 9x9 矩陣位置的關係請參考下圖(即 Pi 下標)：

P ₁	P ₂	P ₃	P ₄	P ₅	P ₆	P ₇	P ₈	P ₉
P ₁₀	P ₁₁	P ₁₂	P ₁₃	P ₁₄	P ₁₅	P ₁₆	P ₁₇	P ₁₈
P ₁₉	P ₂₀	P ₂₁	P ₂₂	P ₂₃	P ₂₄	P ₂₅	P ₂₆	P ₂₇
P ₂₈	P ₂₉	P ₃₀	P ₃₁	P ₃₂	P ₃₃	P ₃₄	P ₃₅	P ₃₆
P ₃₇	P ₃₈	P ₃₉	P ₄₀	P ₄₁	P ₄₂	P ₄₃	P ₄₄	P ₄₅
P ₄₆	P ₄₇	P ₄₈	P ₄₉	P ₅₀	P ₅₁	P ₅₂	P ₅₃	P ₅₄
P ₅₅	P ₅₆	P ₅₇	P ₅₈	P ₅₉	P ₆₀	P ₆₁	P ₆₂	P ₆₃
P ₆₄	P ₆₅	P ₆₆	P ₆₇	P ₆₈	P ₆₉	P ₇₀	P ₇₁	P ₇₂
P ₇₃	P ₇₄	P ₇₅	P ₇₆	P ₇₇	P ₇₈	P ₇₉	P ₈₀	P ₈₁

圖六、POOL 輸入資料矩陣

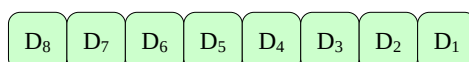
本題採用的 pooling size 為 3x3，對 9x9 矩陣資料計算完 average pooling 的結果為 3x3 矩陣，合計有 9 筆資料，對應到 IDXq 的關係請參考下圖(IDXq 即 Q_i 下標)：

Q ₁	Q ₂	Q ₃
Q ₄	Q ₅	Q ₆
Q ₇	Q ₈	Q ₉

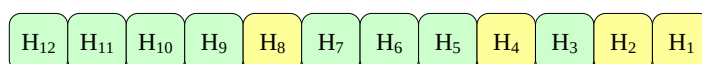
圖七、POOL 計算結果

2.3.5 Hamming code

本題目考慮的原始資料為 8-bit 正整數 D，其二進位表示法如下圖：



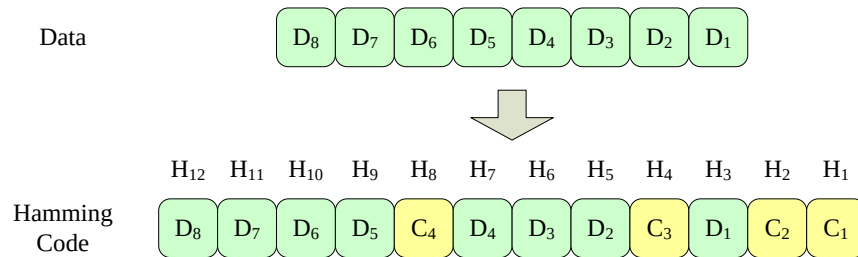
其中 D₁、D₂、D₃、D₄、D₅、D₆、D₇、D₈ 為組成 D 的 8 個 bit，且 D₈ 為 MSB，D₁ 為 LSB。其 Hamming code 為 12 bit H，由 H₁、H₂、H₃、H₄、H₅、H₆、H₇、H₈、H₉、H₁₀、H₁₁、H₁₂ 這 12 個 bit 組成，且 H₁₂ 為 H 的 MSB，H₁ 為 H 的 LSB。



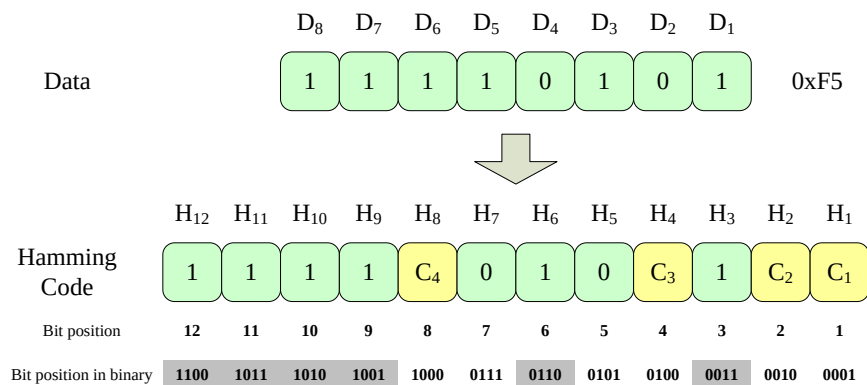
其中 D_1 、 D_2 、 D_3 、 D_4 、 D_5 、 D_6 、 D_7 、 D_8 為組成 D 的 8 個 bit，且 D_8 為 MSB， D_1 為 LSB。其 Hamming code 為 12 bit H ，由 H_1 、 H_2 、 H_3 、 H_4 、 H_5 、 H_6 、 H_7 、 H_8 、 H_9 、 H_{10} 、 H_{11} 、 H_{12} 這 12 個 bit 組成，且 H 的 MSB 為 H_{12} ， H 的 LSB 為 H_1 。

[由 DATA 產生 Hamming code]

Hamming code 中的 H_3 、 H_5 、 H_6 、 H_8 、 H_9 、 H_{10} 、 H_{11} 、 H_{12} 即原始的資料 D ，而 H_1 、 H_2 、 H_4 、 H_7 即 Hamming code 所加入的檢查碼 C ， C 由 C_4 、 C_3 、 C_2 、 C_1 這四個 bit 組成，對應關係如下圖：



其中檢查碼 C 的產生方式直接以實例進行說明。考慮一 8 bit 資料 D 數值為 245 (二進位為 1111_0101)



將資料 D 先填入 Hamming code 對應位置後，挑出其中不為零(即 1 的 bit)的幾個 bit，紀錄下這幾個 bit 位置(指在 Hamming code 中的位置)，將其進行 XOR 計算(即檢查有 1 的個數為奇數還是偶數；奇數為 1，偶數為 0)。上例中，不為零的 bit 為 H_3 、 H_6 、 H_9 、 H_{10} 、 H_{11} 、 H_{12} ，即不為零的 bit 位置為 3、6、9、10、11、12

Non-zero bit	Bit position	Bit position in binary			
H ₁₂	12	1	1	0	0
H ₁₁	11	1	0	1	1
H ₁₀	10	1	0	1	0
H ₉	9	1	0	0	1
H ₆	6	0	1	1	0
H ₃	3	0	0	1	1
		C ₄	C ₃	C ₂	C ₁

即 $C = \{C_4C_3C_2C_1\} = 1100 \oplus 1011 \oplus 1010 \oplus 1001 \oplus 0110 \oplus 0011 = 0001$ ， $\oplus$ 代表 XOR 運算。
 即 $C_4=0$ 、 $C_3=0$ 、 $C_2=0$ 、 $C_1=1$ ，故本例的 Hamming code 為 1111_0010_0101 (0xF25)。

	H ₁₂	H ₁₁	H ₁₀	H ₉	H ₈	H ₇	H ₆	H ₅	H ₄	H ₃	H ₂	H ₁	
Hamming Code	1	1	1	1	0	0	1	0	0	1	0	1	0xF25

[檢查 Hamming code 的正確性及更正錯誤碼]

現假設數值 245 的 Hamming code (0xF25) 在傳輸時，其 H₁₀ 位置的 bit 發生翻轉，造成接收端收到的 Hamming code 為 1101_0010_0101 (0xD25)，此時該如何判斷該 Hamming code 是否正確？且若有 1-bit 錯誤時要如何更正？

	H ₁₂	H ₁₁	H ₁₀	H ₉	H ₈	H ₇	H ₆	H ₅	H ₄	H ₃	H ₂	H ₁	
Hamming Code	1	1	0	1	0	0	1	0	0	1	0	1	0xD25
Bit position	12	11	10	9	8	7	6	5	4	3	2	1	
Bit position in binary	1100	1011	1010	1001	1000	0111	0110	0101	0100	0011	0010	0001	

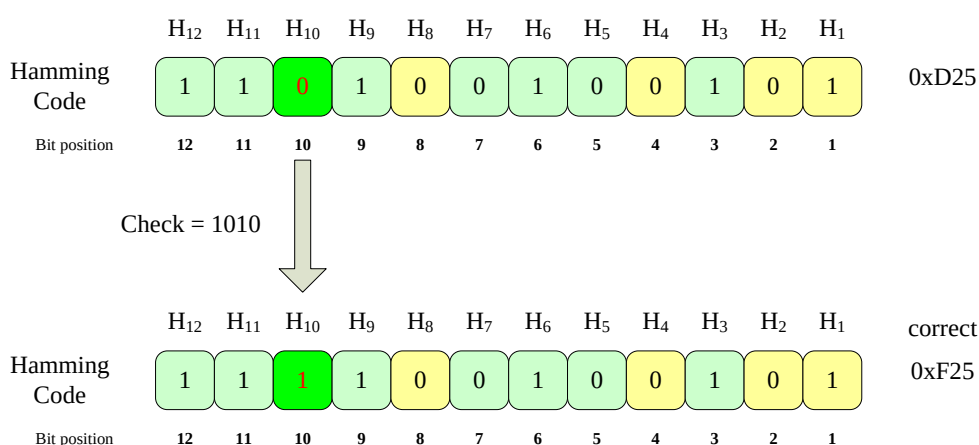
<接收到的 Hamming code 為 0xD25>

將接收到的 Hamming code 中，連同 D 跟 C 兩部分，不為零的 bit 位置記錄起來，並仿產生 Hamming code 一段的方式，將這些不為零的 bit 位置進行 XOR 運算，其計算結果若為 0 代表該 Hamming code 沒有錯誤，若其計算結果不為 0，則該 XOR 結果的值即代表發生 bit 翻轉的位置。本例中不為零的 bit 有 H₁、H₃、H₆、H₉、H₁₁、H₁₂，整理如下進行 XOR 計算：

<i>XOR</i>			Non-zero bit	Bit position	Bit position in binary			
<i>x</i>	<i>y</i>	<i>x</i> ⊕ <i>y</i>						
0	0	0			1	1	0	0
0	1	1	H ₁₂	12	1	0	1	1
1	0	1	H ₁₁	11	1	0	0	1
1	1	0	H ₉	9	0	1	1	0
			H ₆	6	0	0	1	1
			H ₃	3	0	0	0	1
			H ₁	1	1	0	0	1
					C ₄	C ₃	C ₂	C ₁
					1	0	1	0

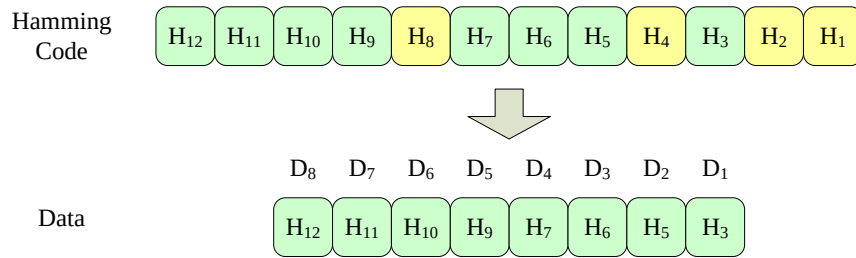
$C = \{C_4C_3C_2C_1\} = 1100 \oplus 1011 \oplus 1001 \oplus 0110 \oplus 0011 \oplus 0001 = 1010$ 。即 $C_4=0$ 、 $C_3=0$ 、 $C_2=0$ 、 $C_1=1$ ，故本例的 Hamming code 為 1111_0010_0101 (0xF25)。該計算的值為 10，代表接收到的 Hamming code 0xD25 有一個 bit 發生翻轉，發生在 H_{10} 的位置；將 H_{10} 位置的 bit 翻轉，得到 0xF25 即為修復後的 Hamming code。若計算結果 $C=0$ 則代表被檢查的 Hamming code 是正確的。

修正的方法很簡單，就是將發生翻轉的 bit 再翻轉一次就可得到正確的 Hamming code。



[由 Hamming code 還原 Data]

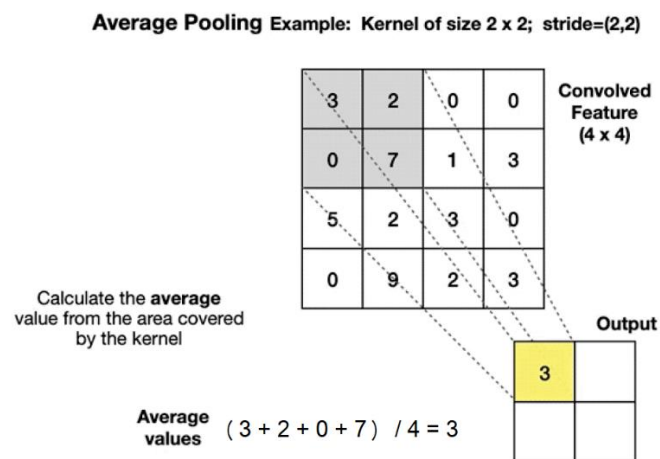
如前面所言，可以從 Hamming code 中的 H_3 、 H_5 、 H_6 、 H_8 、 H_9 、 H_{10} 、 H_{11} 、 H_{12} 即抽出即可組合回原始的資料 D，如下圖：



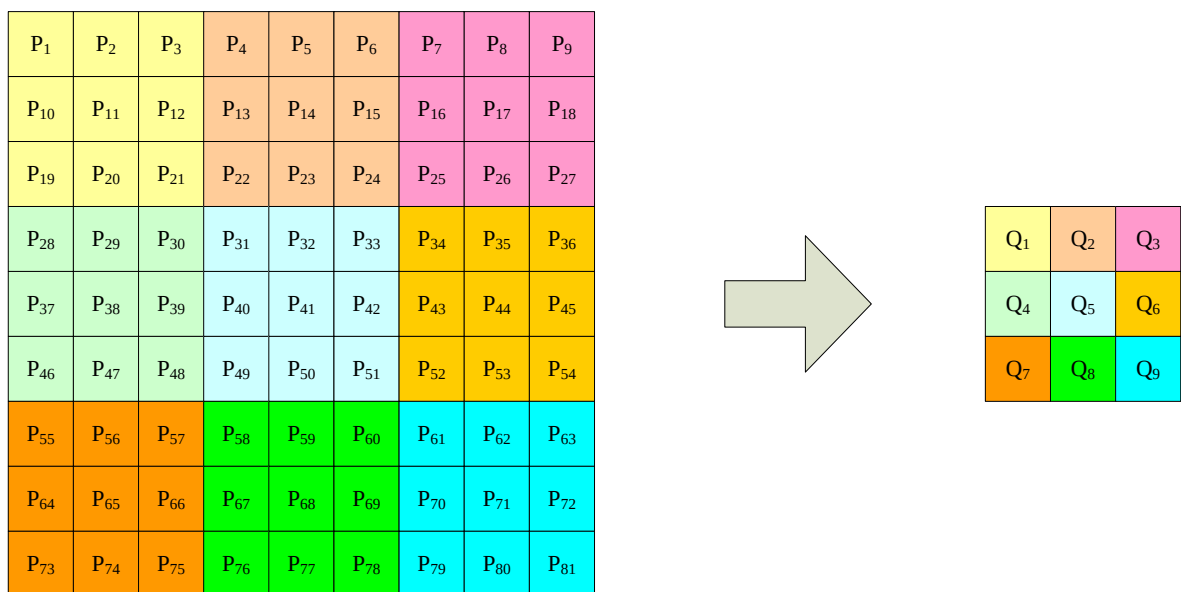
2.3.6 Average Pooling

常見的 pooling 有 max pooling 及 average pooling，本題中採用的是 average pooling。

下圖以一 4x4 矩陣進行 average pooling 到 2x2 矩陣的說明其計算方式：



圖中 $3 = (3+2+0+7) / 4$



以圖中 Q₁ 的計算方式為例

$$Q_1 = (P_1 + P_2 + P_3 + P_{10} + P_{11} + P_{12} + P_{19} + P_{20} + P_{21}) / 9$$

其餘位置的計算方式類推。

上面的除法計算若有產生小數部分，以**直接捨去**的方式處理。

3.評分標準

本題評分方法為：(五組 pattern 總模擬 cycle 數) x 合成面積

- 模擬結束會顯示模擬 DONE cycle：

```
# Detect valid is High. || IDXq = 5, expected 5 :: PASS || Q = 0x066, expected 0x066 :: PASS
# Detect valid is High. || IDXq = 6, expected 6 :: PASS || Q = 0x8ee, expected 0x8ee :: PASS
# Detect valid is High. || IDXq = 7, expected 7 :: PASS || Q = 0x62c, expected 0x62c :: PASS
# Detect valid is High. || IDXq = 8, expected 8 :: PASS || Q = 0x198, expected 0x198 :: PASS
# Detect valid is High. || IDXq = 9, expected 9 :: PASS || Q = 0x007, expected 0x007 :: PASS
#
# DONE cycle found.    valid is 0 (good)    IDXq is 0 (good)    Q is 0 (good)
# DONE cycle at 9
VSIM 109>
```

- 合成面積：

Design compile report area 範例：

dc_shell> report_area

```
Combinational area:          41975.005127
Buf/Inv area:                2082.709759
Noncombinational area:      8700.872564
Macro/Black Box area:       0.000000
Net Interconnect area:     384301.939301
```

```
Total cell area:            50675.877691
Total area:                  434977.816992
```

- STA 分析：

dc_shell> report_timing -sign 5

timing report 中 slack 為正數代表有滿足 timing 規格。

U5237/Y (0AI21X4)	0.12858	6.25865 r
Q7_reg[1]/D (DFFHQX4)	0.00000	6.25865 r
data arrival time		6.25865
clock CLK (rise edge)	6.00000	6.00000
clock network delay (ideal)	0.50000	6.50000
clock uncertainty	-0.10000	6.40000
Q7_reg[1]/CK (DFFHQX4)	0.00000	6.40000 r
library setup time	-0.14063	6.25937
data required time		6.25937

data required time		6.25937
data arrival time		-6.25865

slack (MET)		0.00072

本題評分使用 CBDK_IC_Contest_v2.5 版本，**slow case** library，請勿使用錯誤的 cell library 或是 corner，以免造成無法評分。

本題設定 clock 週期時間為 6ns，參賽者不可調整 clock 週期時間。

◇ 等級 A：

- 在 clock 週期為 6ns 環境下，通過 STA 分析($\text{slack} \geq 0$)且可完成 Gate-Level 與 RTL 模擬，且結果正確。

等級 A 之評分方法：**Score = cycle x Area**

◇ 等級 B：

- 在 clock 週期為 6ns 環境下，模擬有錯。
 - 在調整 clock 週期(超過 6ns)，可完成 Gate-Level 與 RTL 模擬，且結果正確。
- 此時應在 Report Form 上記錄所採用的 clock 周期大小(單位 **ns**，紀錄到**小數下一位**)

等級 B 之評分方法：**Score = (模擬正確之 clock period) x cycle x Area**

◇ 等級 C：

- (1) RTL 模擬結果正確。(2)已完成合成，Gate-Level 可模擬但有部分錯誤。
- 此時以 RTL 模擬之 DONE cycle 數進行評分

等級 C 之評分方法：**Score = cycle**

◇ 等級 D：

- (1) RTL 模擬結果正確。(2)無法合成或 Gate-Level 無法模擬
- 此時以 RTL 模擬之 DONE cycle 數進行評分

等級 D 之評分方法：**Score = cycle**

◇ 等級 E：

- RTL 模擬有部分錯誤。

等級 E 之評分方法：**以錯誤數量作本等級名次排序**

評分方式依設計完成程度，分成 A、B、C、D、E 五種等級，排名順序為 A>B>C>D>E。各等級中，皆以 **Score 越小者** 名次越佳。

附錄

附錄一為設計檔案說明；附錄二為測試樣本資料；附錄三為評分用檔案，亦即參賽者必須繳交的檔案資料；附錄四則為設計檔案壓縮整理步驟說明；附錄五為本次競賽使用之設計資料庫；附錄六為本次競賽使用 EDA 軟體及編輯程式清單。

附錄一 設計檔案說明

1 下表為本題之設計檔案

表 2、設計檔案說明

檔名	說明
POOL.v	請使用本檔案進行設計，檔案內已包含 POOL 模組及輸出入埠宣告。
tb.sv	Test Bench 檔案。
P*.txt, Q*.txt	Test Pattern 及 golden pattern 資料, * = 1~5
dc_syn.tcl	dc_shell 合成參考指令
.synopsys_dc.setup synopsys_dc.setup	使用 Design Compiler 做合成之初始化設定檔。參賽者請依 Library 實際擺放位置，自行修改 Search Path 的設定。注意：合成時請使用 worst case library。
POOL.sdc	合成之 constraint 檔案。
dc_syn.tcl	dc 合成參考指令
xrun.cmd	xrun 模擬參考指令
vcs.cmd	vcs 模擬參考指令
vsim.cmd	vsim 模擬參考指令

2 請使用 **POOL.v** 進行設計。其模組名稱、輸出/入埠宣告如下所示：

```
module POOL(CLK,RST,strobe,IDXp,P,READ,valid,IDXq,Q,DONE);  
  
    input        CLK;  
    input        RST;  
    input        strobe;  
    input [6:0]  IDXp;  
    input        P;  
    output reg    READ;  
    output reg    valid;  
    output reg [3:0]  IDXq;  
    output reg [11:0] Q;  
    output reg    DONE;  
  
endmodule
```

3 本題所提供的 testbench 檔案，有幾行定義如下：

```
`define SDFFILE      "./POOL_syn.sdf"
`ifdef SDF
    initial $sdf_annotate (`SDFFILE, u_POOL);
`endif
```

3.1 SDF 檔之檔名，請自行依 SDF 實際檔名及路徑進行修改後再模擬。

3.2 在進行 gate-level 模擬時，需於模擬命令上多加 +define+SDF 參數才能順利模擬，範例如下

```
xrun -sv tb.sv POOL_syn.v -v tsmc13_neg.v +define+SDF
```

4 本題並有五組測試樣本，模擬時 default 使用測試樣本 P1，若要模擬其它測試樣本，請使用 +define+P2、+define+P3、+define+P4 或 +define+P5 參數。範例如下

```
xrun -sv tb.sv POOL_syn.v -v tsmc13_neg.v +define+SDF+P2
```

5 MAX_CYCLE 預設 5 萬個 cycles，目的是防止因設計電路有錯，模擬陷入無窮迴圈，參賽者可視需要請自行加大 MAX_CYCLE 定義。

6 testbench(tb.sv)以 system verilog 格式撰寫，使用 xrun 模擬時請加入 -sv 參數；使用 vcs 模擬時請加入 -sverilog 參數。

7 RTL 及 Gate-level 模擬的指令說明如下

使用 P1 測試樣本之 RTL simulation 之指令：

➤ 使用 xrun 模擬指令(xrun.cmd)

```
xrun -sv tb.sv POOL.v +define+P1 +access+r -clean
```

➤ 使用 vcs 模擬指令(vcs.cmd)

```
vcs -R -sverilog tb.sv POOL.v +define+P1 +access+r +vcs+fsdbon +fsdb+mda
+fsdbfile+POOL.fsdb
```

➤ 使用 modelsim 模擬，請參考 vsim.cmd 內指令：

```
vsim -c -do vsim.cmd
```

vsim.cmd 內容：

```
vlib work
vlog tb.sv POOL.v +define+P1
vsim work.tb
run -All
```

使用 P1 測試樣本之 Gate-level simulation 之指令：

- 使用 xrun 模擬指令(**xrun.cmd**)

```
xrun -sv tb.sv POOL_syn.v -v tsmc13_neg.v +define+P1+SDF +access+r -clean
-maxdelays
```

- 使用 vcs 模擬指令(**vcs.cmd**)

```
vcs -R -sverilog tb.sv POOL_syn.v -v tsmc13_neg.v +define+P1+SDF +access+r
+vcs+fsdbon +fsdb+mda +fsdbfile+POOL.fsdb +maxdelays +neg_tchk
```

- 使用 modelsim 模擬，請參考 **vsim.cmd** 內指令：

```
vsim -c -do vsim.cmd
```

vsim.cmd 內容：

```
vlib work
vlog tb.sv POOL_syn.v -v tsmc13_neg.v +define+P1+SDF
vsim work.tb
run -All
```

8 以 dc_shell 進行 synthesis 及 STA 分析，參考指令下法說明如下

```
dc_shell -f dc_syn.tcl
```

合成結束會連同面積報告(area.log)及 timing 報告(timing.log)寫出。

附錄二 測試樣本

1. 測試檔案中提供五份測試樣本

測試樣本 P1.txt 及結果 Q1.txt (左側為行號，右邊為 16 進位數值)

P1.txt

1 0a8	26 048	51 046	76 19a
2 aa8	27 04d	52 cee	77 1d8
3 288	28 41f	53 8ee	78 198
4 c8f	29 21f	54 8ef	79 007
5 88d	30 21f	55 66c	80 005
6 8af	31 062	56 6ac	81 017
7 05c	32 046	57 62c	
8 04c	33 0e6	58 198	
9 04c	34 8ee	59 998	
10 2a8	35 aee	60 198	
11 2a8	36 8ec	61 027	
12 2a8	37 60f	62 027	
13 88d	38 61f	63 007	
14 88f	39 69f	64 63c	
15 88d	40 166	65 628	
16 44c	41 046	66 62c	
17 05c	42 067	67 19c	
18 24c	43 8ee	68 190	
19 2a0	44 8ee	69 1d8	
20 2a0	45 8ef	70 007	
21 2aa	46 41f	71 007	
22 8cf	47 21f	72 017	
23 88f	48 63f	73 6ac	
24 88b	49 0e6	74 62e	
25 05c	50 166	75 42c	

Q2.txt

1 2a8
2 88f
3 04c
4 61f
5 066
6 8ee
7 62c
8 198
9 007

測試樣本 P2.txt 及結果 Q2.txt

P2.txt

1 266	26 361	51 8c9	76 c24
2 866	27 364	52 b44	77 c66
3 046	28 cef	53 f44	78 c26
4 3b3	29 aef	54 f45	79 a58
5 7b1	30 aef	55 c78	80 a5a
6 793	31 8ed	56 cb8	81 a48
7 375	32 8c9	57 c38	
8 365	33 869	58 c26	
9 365	34 f44	59 426	
10 066	35 d44	60 c26	
11 066	36 f46	61 a78	
12 066	37 eff	62 a78	
13 7b1	38 eef	63 a58	
14 7b3	39 e6f	64 c28	
15 7b1	40 9e9	65 c3c	
16 765	41 8c9	66 c38	
17 375	42 8e8	67 c22	
18 165	43 f44	68 c2e	
19 06e	44 f44	69 c66	
20 06e	45 f45	70 a58	
21 064	46 cef	71 a58	
22 7f3	47 aef	72 a48	
23 7b3	48 ecf	73 cb8	
24 7b7	49 869	74 c3a	
25 375	50 9e9	75 e38	

Q2.txt

1 066
2 7b3
3 365
4 eef
5 8e9
6 f44
7 c38
8 c26
9 a58

測試樣本 P3.txt 及結果 Q3.txt

P3.txt

1 0a8	26 c22	51 f31	76 8f5
2 08f	27 562	52 cda	77 f03
3 06c	28 9ed	53 484	78 bbf
4 21f	29 6e4	54 dc0	79 976
5 064	30 22c	55 78b	80 561
6 8ce	31 61b	56 844	81 547
7 63c	32 072	57 edc	
8 198	33 808	58 bb8	
9 007	34 282	59 438	
10 066	35 858	60 a72	
11 7b3	36 96d	61 bca	
12 365	37 92a	62 2f0	
13 eed	38 19f	63 055	
14 8e9	39 0e6	64 797	
15 f46	40 41c	65 8a1	
16 838	41 674	66 4fc	
17 c36	42 da6	67 352	
18 858	43 019	68 359	
19 eac	44 a72	69 7b8	
20 7bc	45 1e6	70 1ac	
21 19a	46 e74	71 f0f	
22 8b7	47 69c	72 dd6	
23 f11	48 072	73 698	
24 c77	49 cd9	74 4e0	
25 ed2	50 262	75 b76	

Q3.txt

1 505
2 a14
3 8bb
4 52f
5 7ff
6 667
7 7ff
8 942
9 7b3

測試樣本 P4.txt 及結果 Q4.txt

P4.txt

1 ac4	26 8f4	51 058	76 4b2
2 cc4	27 969	52 9eb	77 06a
3 daa	28 d5a	53 95b	78 f3c
4 9be	29 6b1	54 f6f	79 64d
5 507	30 39e	55 c55	80 ba3
6 91a	31 a10	56 5b1	81 8d3
7 071	32 2dd	57 4fb	
8 c12	33 b15	58 635	
9 c40	34 57d	59 852	
10 7b4	35 6a9	60 bb8	
11 365	36 f3e	61 368	
12 198	37 c4e	62 7ec	
13 7fa	38 eba	63 bc0	
14 667	39 5d0	64 933	
15 34d	40 3ce	65 913	
16 5fe	41 ecf	66 02a	
17 d84	42 dd9	67 54a	
18 aa5	43 62c	68 dd7	
19 7c3	44 ba6	69 4a5	
20 55f	45 df4	70 dc6	
21 4cd	46 5cc	71 667	
22 33c	47 0a9	72 7a3	
23 2d0	48 daa	73 cd9	
24 54d	49 250	74 bba	
25 8ab	50 b5f	75 b24	

Q4.txt

1 62b
2 635
3 8bb
4 917
5 7d5
6 a5f
7 8a2
8 67e
9 8c3

測試樣本 P5.txt 及結果 Q5.txt

P5.txt

1 0c9	26 bbc	51 4db	76 8b9
2 ae4	27 7e7	52 e41	77 fla
3 clf	28 feb	53 a46	78 fl1
4 6fa	29 cbb	54 a0c	79 31a
5 049	30 3cb	55 038	80 bef
6 1c0	31 7fc	56 e3a	81 8b2
7 f7e	32 dff	57 976	
8 2ce	33 cf4	58 2d0	
9 1e0	34 7cc	59 ad0	
10 8bb	35 d5a	60 8ee	
11 dd8	36 604	61 951	
12 365	37 1a5	62 368	
13 630	38 a0d	63 7cc	
14 780	39 83b	64 c50	
15 79c	40 402	65 c5d	
16 fd9	41 f05	66 19f	
17 56a	42 32f	67 650	
18 d5a	43 2ce	68 c37	
19 daf	44 c0c	69 bf8	
20 1db	45 c27	70 bd9	
21 bc5	46 b42	71 c21	
22 377	47 0d6	72 bd7	
23 93a	48 7be	73 990	
24 c0f	49 e68	74 ba3	
25 2a1	50 9da	75 5ff	

Q5.txt

1 7e1
2 54e
3 8a5
4 7f8
5 945
6 95c
7 8ee
8 923
9 8ee

附錄三 評分用檔案

評分所須檔案可以下幾個部份：

- (1) RTL design，各參賽隊伍對該次競賽設計的 RTL code，
為方便評分，若設計採模組化而有多個檔案，繳交 RTL 時請將所有檔案合併至 POOL.v 中，只繳交一個 POOL.v 檔案。
- (2) Gate-Level design，由合成軟體所產生的 gate-level netlist，以及對應的 SDF 檔；
- (3) 參賽隊伍須依照自己的設計內容，填寫「Design Report Form」，減少評分出錯機會。

表 3、繳交檔案

<i>RTL category</i>		
<i>Design Stage</i>	<i>File</i>	<i>Description</i>
N/A	N/A	Design Report Form (繳交紙本)
RTL Simulation	POOL.v or POOL.sv	Verilog (or System Verilog) synthesizable RTL code
<i>Gate-Level category</i>		
<i>Design Stage</i>	<i>File</i>	<i>Description</i>
Pre-layout Gate-level Simulation	POOL_syn.v	Verilog gate-level netlist generated by Synopsys Design Compiler
	POOL_syn.sdf	SDF timing information generated by Synopsys Design Compiler
	POOL_syn.ddc	Design database generated by Synopsys Design Compiler

附錄四、檔案整理步驟

當所有的文件準備齊全如表 3 所列，請按照以下的步驟指令，提交相關設計檔案，將所有檔案複製至同一個資料夾下，步驟如下：

1. 在自己的 home directory 建立一個新目錄，名稱叫做“**result**” 例如：
 > **mkdir ~/result**
2. 將附錄 D 要求的檔案複製到 result 這個目錄。例如：
 > **cp POOL.v ~/result/**
 > **cp POOL_syn.v ~/result/**
 其他檔案依此類推.....
3. 若所設計電路包含多個模組，請將所有電路模組放到同一個 POOL.v (或 POOL.sv) 內，且 POOL 模組須放在第一個。
4. 在 Design Report Form(紙本表單)中，填入所需的相關資訊。